Prompt Registry & Local LLM Inference with MLflow 3.13

Versioning, aliasing and evaluating prompts with Llama 3.2 on Google Colab Pro

Agenda

Part 1 — Prompt Registry

- Why prompts need governance
- Register, load, version
- Aliases: promote and roll back
- Compare and evaluate versions

Part 2 — Our implementation

- Why we run the model locally
- Ollama + Llama 3.2 on a Colab T4
- Typed output with Pydantic AI
- Results, limitations, lessons

> Base material: the MLflow 3.13 *Prompt Registry & AI Gateway* workshop. **Our change:** the OpenAI/Claude calls are replaced with a local open-weights model.

Part 1

Prompt Registry

Treat prompts like code and models: versioned, reviewable, reversible.

Before: the prompt lives next to the code

```
SYSTEM_PROMPT = """
Classify this support ticket into one of:
PAYMENT, REFUND, RESCHEDULE, or OTHER.
"""

def triage(ticket: str) -> str:
    response = client.chat.completions.create(
        model="llama3.2:3b",
        messages=[{"role": "system", "content": SYSTEM_PROMPT},
                  {"role": "user", "content": ticket}],
    )
    return response.choices[0].message.content
```

Fine for a prototype. **It breaks down in a real project.**

The problems

Problem	What it looks like in practice
No version history	You "improve" a prompt, quality drops, and you cannot recover the old wording.
Deployment coupling	A one-word tweak needs a code change, PR, build and redeploy.
No safe rollback	Reverting means reverting a commit, not flipping a pointer.
Hard to compare	"What changed between the version that worked and the one that didn't?"
No lineage	Which prompt version produced this output or evaluation run?
No quality loop	Changes are judged by gut feeling, not by metrics on a dataset.

> **Root cause:** prompts are treated as *incidental strings* rather than *versioned artifacts*.

What the MLflow Prompt Registry offers

- **Version control** — every version is **immutable**, with a commit message
- **Aliases** — mutable pointers such as `production` to promote and roll back **without code changes**
- **Templates** — `{{ variables }}` keep instructions separate from input data
- **Lineage & evaluation** — plugs into `mlflow.genai.evaluate()` and MLflow tracking
- **Central UI** — browse, diff and manage prompts in one place

All APIs live under **`mlflow.genai`**.

Core concepts

Concept	Meaning
Prompt	A named, versioned template
Template	Text with `{{ variables }}` (Jinja2-style)
Version	Sequential and immutable (`v1`, `v2`, ...)
Alias	Mutable pointer, e.g. `@production`
URI	`prompts:/name/2` (exact) or `prompts:/name@production` (alias)

```
mlflow.genai.register_prompt(name=..., template=..., commit_message=...)
mlflow.genai.load_prompt("prompts:/workshop-ticket-triage@production")
mlflow.genai.set_prompt_alias(name, alias="production", version=2)
```

Part 2

Our implementation

Same governance workflow, but the model runs on our own GPU.

Why we changed the model backend

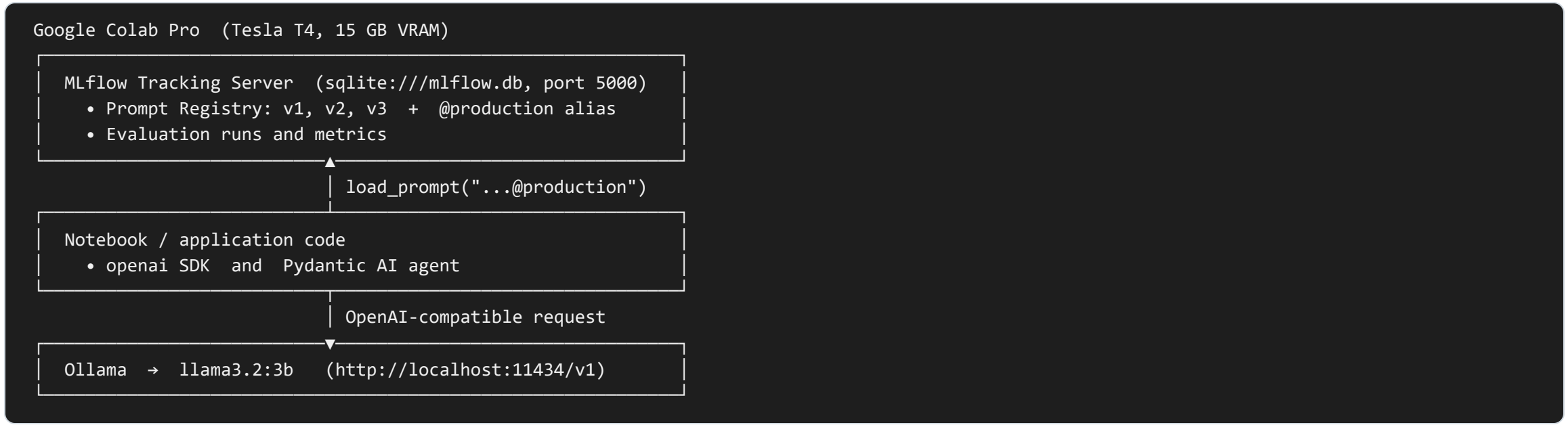
What we hit with cloud APIs

- No OpenAI / Claude key available
- Gemini free tier returned **HTTP 429**: limit of **5 requests / minute** per model
- Evaluation runs many calls in a burst, so quota is exhausted quickly
- `mlflow gateway start` is **not supported on Windows**

What we did instead

- Run **Llama 3.2 (3B)** with **Ollama** on a **Colab Pro T4 GPU**
- Ollama exposes an **OpenAI-compatible** API at `localhost:11434/v1`
- Same `openai` SDK code, only `base_url` and `model` change
- No API key, no rate limit, data stays in the session

Architecture



Setup: Ollama on Colab

```
apt-get install -y -qq pciutils zstd          # zstd is needed to unpack Ollama
curl -fsSL https://ollama.com/install.sh | sh
ollama serve &                                # start the daemon
ollama pull llama3.2:3b                       # download the model
```

```
from openai import OpenAI

local_llm_client = OpenAI(
    base_url="http://localhost:11434/v1",
    api_key="ollama",            # placeholder; the SDK requires a value
)
```

> **Gotcha we hit:** the first install failed with `requires zstd` until it was installed.
> **GPU choice:** a standard **T4** is enough for a 3B model.

Step 1 — Register version 1

```
V1_TEMPLATE = """Classify this support ticket into one category:
PAYMENT, REFUND, RESCHEDULE, or OTHER.

Ticket: {{ ticket }}

Respond with the label only."""

prompt_v1 = mlflow.genai.register_prompt(
    name="workshop-ticket-triage",
    template=V1_TEMPLATE,
    commit_message="v1: Initial minimal classification prompt",
    tags={"task": "ticket-triage", "backend": "colab-gpu"},
)
# Registered workshop-ticket-triage version 1
# URI: prompts:/workshop-ticket-triage/1
```

The application fills the variable at call time: `loaded.format(ticket=...)`.

Step 2 — Version 2 and a Git-style diff

```
V2_TEMPLATE = """You are an expert support
ticket triage assistant.

Task: Classify the ticket into
exactly one of:
- PAYMENT: billing, duplicate charges
- REFUND: money back, cancellations
- RESCHEDULE: move appointment/date
- OTHER: general inquiries

Rules:
- Output exactly one uppercase label
- No explanation or punctuation

Ticket: {{ ticket }}"""
```

```
--- v1
+++ v2
-Classify this support ticket...
-PAYMENT, REFUND, RESCHEDULE...
+You are an expert support ticket
+triage assistant.
+Task: Classify the ticket into
+exactly one of:
+- PAYMENT: billing, ...
+- REFUND: money back, ...
+Rules:
+- Output exactly one uppercase label
 Ticket: {{ ticket }}
-Respond with the label only.
```

Produced with `difflib.unified_diff` on the two loaded templates (the UI has a Compare view too).

Step 3 — Promote with an alias

```python
mlflow.genai.set_prompt_alias(
    "workshop-ticket-triage",
    alias="production",
    version=2,
)

prod = mlflow.genai.load_prompt(
    "prompts:/workshop-ticket-triage@production"
)
# alias @production -> version 2
```

Why it matters

- App code references `@production`, never a version number
- Promote = move the pointer
- **Roll back** = move it back (`version=1`)
- No rebuild, no redeploy

> Alias lookups are cached briefly, so a change may take a moment to be seen by running apps.

Step 4 — Governed prompt + local model

```python
def classify_ticket(ticket, prompt_uri=None):
    uri = prompt_uri or "prompts:/workshop-ticket-triage@production"
    prompt = mlflow.genai.load_prompt(uri)
    response = local_llm_client.chat.completions.create(
        model="llama3.2:3b",
        messages=[{"role": "user", "content": prompt.format(ticket=ticket)}],
        temperature=0.1,
    )
    return response.choices[0].message.content.strip()
```

Ticket (output of our run)	Label
I was charged twice for one hotel night.	PAYMENT
Need to move my dental appointment to next Thursday.	RESCHEDULE
I want my money back for a cancelled flight.	REFUND
What are your customer support working hours?	OTHER

Step 5 — Typed output with Pydantic AI

```
class TriageResult(BaseModel):
    category: Literal["PAYMENT", "REFUND", "RESCHEDULE", "OTHER"]
    reason: str

pydantic_model = OpenAIChatModel(
    "llama3.2:3b",
    provider=OpenAIProvider(base_url="http://localhost:11434/v1", api_key="ollama"),
)

agent = Agent(pydantic_model, output_type=TriageResult,
              system_prompt="Classify customer support tickets accurately.")
result = agent.run_sync(prompt.format(ticket="I was charged twice for one hotel night."))
```

```
Category : PAYMENT
Reason   : duplicate charges
```

The registry owns the **prompt**, the local server owns the **model**, Pydantic AI owns the **output schema**.

Step 6 — Evaluate versions on a dataset

```
EVAL_DATA = [{"inputs": {"ticket": "Charged twice for one hotel night."},
              "expectations": {"expected_response": "PAYMENT"}}, ...]   # 4 rows

@scorer
def category_match(outputs, expectations):          # contains the expected label?
    return float(expectations["expected_response"] in str(outputs).strip().upper())

@scorer
def label_guidelines(outputs):                      # is it exactly one valid label?
    return float(str(outputs).strip().upper().strip(".") in
                 {"PAYMENT", "REFUND", "RESCHEDULE", "OTHER"})

with mlflow.start_run(run_name="eval-v1-local"):
    mlflow.genai.evaluate(data=EVAL_DATA, predict_fn=make_predict_fn(uri_v1),
                          scorers=[category_match, label_guidelines])
```

Both scorers are **deterministic** (no LLM judge), so results are repeatable.

Results

Version	`category_match`	`label_guidelines`	Note
v1	**0.50**	1.00	minimal prompt
v2	**1.00**	1.00	explicit labels and rules → promoted
v3	**1.00**	1.00	`@production` re-evaluated

- Adding explicit label definitions and output rules raised `category_match` from **0.50 to 1.00** on our dataset.
- v3 matched v2; **the dataset cannot separate them**.
- v3 was written **by hand** as a refinement. We did not run `optimize_prompts()`.

Limitations (what we should not over-claim)

- **Tiny dataset:** 4 examples, one per label. Good for a demo, not a statistical result.
- **Weak scorers:** `category_match` accepts the label anywhere in the text, so it is lenient.
- **Small model:** a 3B model is more sensitive to prompt wording; results may differ for larger models.
- **No AI Gateway in this run:** routing, fallbacks and usage tracking were covered conceptually; we called Ollama directly.
- **Session storage:** the MLflow SQLite DB lives in the Colab session and is lost on disconnect unless saved to Drive.

Challenges and lessons learned

Challenge	How we handled it
Gemini free tier: HTTP 429 (5 req/min)	Moved inference to a local model
`mlflow gateway start` fails on Windows	Ran everything on Colab (Linux)
Ollama install: `requires zstd`	`apt-get install zstd` first
Notebook ran before the MLflow server was ready	Wait for the server, then re-run the setup cell
Colab disconnects lose state	Optional Google Drive-backed SQLite store

Summary

Prompt Registry — *what you say*

- Immutable versions with commit messages
- Aliases decouple prompts from deploys
- Diff, evaluate, promote, roll back

Local inference — *how we ran it*

- Llama 3.2 (3B) via Ollama on a Colab T4
- OpenAI-compatible API, so code barely changes
- No API key or rate limit during evaluation

> **Take-away:** prompt changes become measurable, reversible and decoupled from deployment.

References

- MLflow Prompt Registry — https://mlflow.org/docs/latest/genai/prompt-registry/
- Evaluate & monitor GenAI — https://mlflow.org/docs/latest/genai/eval-monitor/
- MLflow AI Gateway — https://mlflow.org/docs/latest/genai/governance/ai-gateway/
- Ollama — https://ollama.com/ · Llama 3.2 — https://ollama.com/library/llama3.2
- Pydantic AI — https://ai.pydantic.dev/
- Source repository — https://github.com/gromx-log/prompt_registry_and_ai_gateway_using_mlflow_and_local_ollama

Thank you

Questions & discussion